Magik (linguagem de programação)

Magik é uma linguagem de programação orientada a objetos que oferece suporte à herança múltipla e ao polimorfismo, além de possuir tipagem dinâmica. Foi projetada e implementada em 1989 por Arthur Chance, da Smallworld Systems Ltd., como parte do Sistema de Informação Geográfica (GIS) Smallworld. Após a aquisição da Smallworld em 2000, a Magik passou a ser fornecida pela GE Energy, continuando como parte da plataforma tecnológica Smallworld.

A Magik (Inspirational Magik) foi apresentada originalmente em 1990 e recebeu melhorias e atualizações ao longo dos anos. Sua versão atual é a 5.2.

Em julho de 2012, desenvolvedores da Magik anunciaram que estavam trabalhando na migração da linguagem para a máquina virtual Java. A conclusão bem-sucedida dessa migração foi confirmada pela Oracle Corporation em novembro do mesmo ano.[1]

Semelhanças com Smalltalk

A própria Magik compartilha algumas semelhanças com Smalltalk em relação aos recursos da linguagem e à sua arquitetura: o código Magik é compilado em bytecodes, que são interpretados pela máquina virtual Magik. Essa máquina virtual está disponível em diversas plataformas, incluindo Microsoft Windows e diferentes versões de Unix e Linux.

A Magik é baseada em console, e o código pode ser modificado em tempo real, mesmo enquanto uma aplicação está em execução. O console também pode ser usado para executar código Magik e visualizar os resultados.

O código compilado é armazenado em um único arquivo chamado arquivo de imagem (image file). Cada arquivo de imagem contém os bytecodes compilados e o estado da sessão no momento em que a imagem foi salva pela última vez (por exemplo, os valores das variáveis).

Recursos da linguagem

Comentários

A Magik usa o símbolo # para marcar trechos do código como comentários:

```
# Este é um comentário.
```

Atribuições

A Magik usa o operador << para realizar atribuições:

```
a << 1.234
b << b + a
c << "foo" + "bar" # Concatenar strings
```

Para facilitar a compreensão, essa notação é lida como “a passa a ser 1.234” ou “b passa a ser b mais a”. Essa terminologia diferencia a atribuição da comparação.

A Magik também oferece uma variação compacta desse operador, que funciona de maneira semelhante aos operadores encontrados na linguagem C:

```
b +<< a # Equivalente a b << b + a
```

Para exibir o valor de uma variável, você pode usar o seguinte comando:

```
a << "hello"
write(a)
```

Símbolos

Além dos tipos de dados convencionais, como inteiros, números de ponto flutuante e strings, a Magik também implementa símbolos. Símbolos são um tipo especial de dado usado extensivamente na Magik para identificar

objetos de forma única. Eles são representados por dois-pontos seguidos de uma sequência de caracteres. É possível delimitar símbolos que contenham espaços usando o caractere de barra vertical. Por exemplo:

```
a << :hello # sempre que :hello for encontrado, será a mesma instância
b << :|hello world|
```

Tipagem dinâmica

As variáveis Magik não possuem tipos fixos, como acontece, por exemplo, em C#, e podem referenciar objetos diferentes durante a execução. Tudo na Magik é um objeto (não existe distinção entre objetos e tipos primitivos, como inteiros):

```
a << 1.2      # um número de ponto flutuante é atribuído à variável 'a'
a << "1.2"    # posteriormente, uma string é atribuída à variável 'a'
```

Objetos

Os objetos são implementados na Magik por meio de exemplares (exemplars). Os exemplares têm semelhanças com as classes de outras linguagens de programação, como Java, mas apresentam diferenças importantes. A Magik oferece suporte à herança múltipla e a mixins (que implementam funcionalidades sem armazenar dados). Novas instâncias são criadas clonando uma instância existente, que normalmente será o exemplar, mas não precisa ser.

Novos exemplares são criados usando a instrução def_slotted_exemplar(), por exemplo:

```
def_slotted_exemplar(:my_object,
  {
    {:slot_a, 34},
    {:slot_b, "hello"}
  }, {:parent_object_a, :parent_object_b})
```

Esse trecho de código define um novo exemplar chamado my_object, que possui dois slots (ou campos): slot_a, pré-inicializado com 34, e slot_b, pré-inicializado com "hello". Esse exemplar herda de dois exemplares existentes chamados parent_object_a e parent_object_b.

Comparação

A Magik implementa os operadores lógicos usuais (=, <, <=, >, >=, ~=/<>) para comparação, além de alguns operadores menos comuns. Os operadores _is e _isnt são usados para comparar instâncias específicas de objetos, ou referências a objetos, em vez de comparar seus valores.

Por exemplo:

```
a << "hello"
b << "hello"

a = b # retorna True (_true), porque os valores de a e b são iguais
a _is b # retorna False (_false), porque a não é a mesma instância que b

a << "hello"
b << a
a = b # retorna True (_true), porque os valores de a e b são iguais
a _is b # retorna True (_true), porque b recebeu a referência à mesma instância de a,
        # e não apenas o valor de a.
```

Métodos

Os métodos são definidos nos exemplares usando as instruções _method e _endmethod:

```
_method my_object.my_method(a, b)
  _return a + b
_endmethod
```

É convenção fornecer dois métodos: new() (para criar uma nova instância) e init() (para inicializar uma instância).

```
# Método de criação
```

```
_method person.new(name, age)
  _return _clone.init(name, age)
_endmethod

# Método de inicialização
_private _method person.init(name, age)
  # Chama a implementação do exemplar pai.
  _super.init(name, age)
  # Inicializa os slots.
  .name << name
  .age << age
  _return _self
_endmethod
```

A instrução _clone cria uma cópia física do objeto person. A instrução _super permite que os objetos invoquem a implementação de um método definida no exemplar pai. Os objetos podem fazer referência a si mesmos usando a instrução _self. Os slots de um objeto são acessados e recebem valores usando a notação de ponto.

Métodos que não fazem parte da interface pública do objeto podem ser marcados como privados usando a instrução _private. Métodos privados só podem ser chamados por _self, _super e _clone.

Argumentos opcionais podem ser declarados usando a instrução _optional. Os argumentos opcionais que não forem informados recebem da Magik o objeto especial _unset (equivalente a null). A instrução _gather pode ser usada para declarar uma lista de argumentos opcionais.

```
_method my_object.my_method(_gather values)
_endmethod
```

Iteração

Na Magik, as instruções _while, _for, _over, _loop e _endloop permitem realizar iterações.

```
_block
  _local s << 0
  _local i << 0
  _while i <= 100
  _loop
    s +<< i
    i +<< 1
  _endloop
  >> s
_endblock
```

Nesse exemplo, a instrução _while é combinada com _loop e _endloop.

```
_method my_object.my_method(_gather values)
  total << 0.0
  _for a _over values.elements()
  _loop
    total +<< a
  _endloop
  _return total
_endmethod

m << my_object.new()
x << m.my_method(1.0, 2, 3.0, 4) # x = 10.0
```

Nesse caso, values.elements() é um iterador que ajuda a percorrer os valores.

Na Magik, métodos geradores são chamados de métodos iteradores. Novos métodos iteradores podem ser definidos usando as instruções _iter e _loopbody:

```
_iter _method my_object.even_elements()
  _for a _over _self.elements()
  _loop
    _if a.even? _is _true
    _then
      _loopbody(a)
    _endif
  _endloop
_endmethod
```

Procedimentos

A Magik também oferece suporte a funções chamadas procedimentos (procedures). Procedimentos também são objetos e são declarados usando as instruções _proc e _endproc. Os procedimentos são atribuídos a variáveis, que podem então ser invocadas:

```
my_procedure << _proc @my_procedure(a, b, c)
  _return a + b + c
_endproc

x << my_procedure(1, 2, 3) # x = 6
```

Expressões regulares

A Magik oferece suporte à sintaxe de expressões regulares //:

```
_if /Hello\,\s(\w)+!/.matches?("Hello, Magik!") _then
  write("Got a match!")
_endif
```

Também é possível capturar grupos em expressões regulares:

```
/sw([0-9]+)-([0-9]+).*/.replace_all("sw65456-324sss", "$1") # "65456"
/sw([0-9]+)-([0-9]+).*/.replace_all("sw65456-324sss", "$2") # "324"
```

Biblioteca HTTP

A Magik permite realizar requisições HTTP ou HTTPS por meio da biblioteca http. Veja os exemplos abaixo:

```
magikhttp << http.new()
magikhttp.url("https://www.google.com").get()
magikhttp.url("https://www.google.com").post({"User-agent", "Bot"}, "some data")
```

Particularidades da linguagem

Como a Magik foi originalmente desenvolvida na Inglaterra, os métodos das bibliotecas principais do Smallworld são escritos usando o inglês britânico. Por exemplo:

```
Use "initialise", not "initialize".
```

Use "initialise", e não "initialize".

Coleções

Assim como outras linguagens de programação, a Magik também possui coleções. Entre elas estão:

```
Vetor simples (Simple Vector)
Rope - um array dinâmico
Tabela hash (Hash Table)
Lista de propriedades (Property List)
Conjunto por igualdade (Equality set)
Bags
```

Exemplo de Hello World

A seguir, um exemplo do programa Hello World escrito em Magik:

```
write("Hello World!")
```

Referências

1. Jim Connors (05/11/2012). “Sprinkle Some Magik on that Java Virtual Machine”. Oracle Blogs. Recuperado em 06/11/2012. O texto informa que, com essa nova capacidade, a GE Energy conseguiu hospedar seu ambiente Magik sobre a máquina virtual Java.

Links externos

Smallworld Product Suite Technology — conjunto de tecnologias do Smallworld.

MDT - Magik Development Tools — ambiente de desenvolvimento integrado (IDE) para desenvolvedores do GE Smallworld GIS.

Open Source (SourceForge) — projeto de código aberto.

Publicação em fórum sobre a linguagem Magik.

Technical Paper No. 5 - An Overview of Smallworld Magik — documento técnico com uma visão geral da Magik do Smallworld.

Extensão do Visual Studio Code para a linguagem de programação Smallworld Magik.